



## Exercice 1 — Créer une API de To-Do List

**Objectif** : Créer une API REST permettant d'ajouter, consulter, modifier et supprimer des tâches.

---



### Objectif global

Créer un serveur Express qui interagit avec MongoDB pour gérer une liste de tâches (To-Do).

---



### Étapes à suivre (guidées)

---



#### Étape 1 – Préparer ton environnement

1. Crée un dossier `todo-api`
2. Initialise un projet Node.js avec `npm init -y`
3. Installe les dépendances :
  - `express`
  - `mongoose`
  - `cors`
  - `dotenv`



Commande : `npm install express mongoose cors dotenv`

---



#### Étape 2 – Démarrer un serveur Express

1. Crée un fichier `index.js`

2. Configure un serveur Express de base
3. Ajoute la gestion du JSON (`express.json()`)
4. Affiche un message dans la console quand le serveur démarre

 Questions :

- Sur quel port ton serveur doit-il écouter ?
- Que se passe-t-il si tu oublies `express.json()` ?

---

### ✅ Étape 3 – Connexion à MongoDB

Crée un fichier `.env` et stocke-y une URI MongoDB comme :

`MONGO_URI=mongodb://localhost:27017/todo_db`

- 1.
2. Utilise `mongoose.connect(...)` pour te connecter à la base de données
3. Vérifie que la connexion est bien réussie dans la console

---

### ✅ Étape 4 – Créer un modèle Mongoose

1. Crée un dossier `models`
2. Crée un fichier `Task.js`
3. Le modèle doit inclure :
  - `title` (String, requis)
  - `completed` (Boolean, par défaut à false)

 Question : Pourquoi définir un `default` dans un schéma Mongoose ?

---

### ✅ Étape 5 – Créer les routes CRUD

1. Crée un dossier `routes`
2. Crée un fichier `tasks.js`
3. Implémente les routes suivantes :
  - `POST /tasks` → Créer une tâche
  - `GET /tasks` → Obtenir toutes les tâches
  - `GET /tasks/:id` → Obtenir une tâche spécifique
  - `PUT /tasks/:id` → Modifier une tâche
  - `DELETE /tasks/:id` → Supprimer une tâche

💡 Astuce : utilise `Task.find()`, `Task.findById()`, `Task.findByIdAndUpdate()`, etc.

---

## ✅ Étape 6 – Brancher les routes

1. Dans `index.js`, importe et utilise les routes `tasks.js` avec un prefix `/tasks`

```
const taskRoutes = require('./routes/tasks');
app.use('/tasks', taskRoutes);
```

---

## ✅ Étape 7 – Tester ton API

- Utilise **Postman** ou **Thunder Client**
  - Crée plusieurs tâches
  - Teste la mise à jour et la suppression
  - Vérifie les erreurs (ex. ID invalide)
- 

## 🔄 Étape bonus – Ajoute un champ "date de création"

- Ajoute un champ `createdAt` dans le schéma
- Rends-le automatique avec `{ type: Date, default: Date.now }`



## Exercice 2 – API de gestion de livres

### Contexte :

Tu dois créer une API REST pour gérer une bibliothèque.

### Fonctionnalités attendues :

- Ajouter un livre (titre, auteur, année)
- Récupérer tous les livres
- Rechercher un livre par titre
- Modifier un livre
- Supprimer un livre

### Contraintes :

- Base MongoDB : library
- Collection : books
- Respecte les bonnes pratiques REST

⚠ Tu dois tout faire sans aide cette fois. Tu peux utiliser Postman ou Thunder Client pour tester ton API.

---



## Exercice 3 – API de contacts avec champs avancés

### Contexte :

Tu développes une API pour une application de gestion de contacts.

### Données :

Chaque contact possède :

- prénom
- nom
- email
- numéro de téléphone
- favori (boolean)

### Fonctionnalités :

- Ajouter un contact
- Lister les contacts
- Lister uniquement les contacts favoris
- Modifier les informations d'un contact
- Supprimer un contact

### Bonus :

- Ajouter une recherche par nom ou email (query param dans l'URL)